開発者向けガイド

本章は Vivlio Starter をフォークして改造・拡張したい開発者に向けた案内です。著者として執筆するだけであれば読む必要はありません。各コマンドの使い方はそれぞれの章にあります。ここでは「どこに何があるか」と「手を入れるならどこを触るか」に絞って述べます。

1 全体像

Vivlio Starter は、Markdown の原稿から閲覧用 PDF・印刷入稿用 PDF・EPUB・Kindle 用 KPF までを作る**電子書籍執筆システム**です。**組版エンジンとして Vivliostyle を採用**し、その前後——原稿の前処理、文章校正、索引・用語集、画像の最適化、表紙の生成、PDF の後処理——を自前で持ちます。六つの領域に分かれた 26 のコマンドが、執筆から入稿までの工程を覆います。

実装は Ruby gem 一つにまとまっており、CLI フレームワークには Samovar を採用しています。

```
bin/vs / bin/vivlio-starter
  └ lib/vivlio_starter/cli/startup.rb   # CLI.start・無効入力時のヘルプ
       └ cli/loader.rb                  # ドメイン + Samovar の一括 require
            └ SamovarCommands::RootCommand
                 └ 各コマンド（build / lint / metrics / ...）

lib/vivlio_starter/cli.rb              # ライブラリからフル CLI を読む場合（startup を経由）
```

`CLI.start(ARGV)` の定義と無効オプション時の `--help` 誘導は `cli/startup.rb` に単一定義されています。ドメインから Samovar までの `require` 順は `cli/loader.rb` に集約されています。

♣ 二層構造

各コマンドは **Samovar エントリ**（`cli/samovar/` 配下）と**ドメイン実装**（`cli/` 配下）の二層に分かれています。前者が引数とオプションの受け取り・ヘルプ表示を担い、後者が実際の処理を持ちます。

```
lib/vivlio_starter/
  cli.rb                    # フル CLI 読み込み
  cli/
    startup.rb              # CLI.start 単一定義
    loader.rb               # require の順序
    common.rb               # CONFIG・ログ・パス定数
    config_keys.rb          # book.yml のキーと既定値の宣言（1 表）
    token_resolver.rb       # 章番号・スラッグの解決
    masking.rb              # コード領域解釈の唯一の実装
    samovar/                # Samovar CLI コマンド定義
      root_command.rb       # コマンドルーティング
      build_command.rb
      ...
    build/                  # ビルドパイプライン
      pipeline.rb           # ステップ表・相・並列実行
      epub_flow.rb          # EPUB / Kindle の生成フロー
      ...
    pre_process/            # Markdown 前処理
    post_process/           # HTML 後処理
    index/                  # 索引・用語集
    metrics/                # メトリクス分析
    lint/                   # 校正補助
    pdf/                    # PDF 読み取り・プロバイダ
    guards/                 # 実行前提条件の検査
```

この分離のおかげで、CLI の見た目を変えずに処理を差し替えたり、処理を変えずにオプションを足したりできます。

♣ コマンドとファイルの対応

置き場所は規則的です。コマンド `foo` に対して、Samovar エントリは `cli/samovar/foo_command.rb`、ドメイン実装は `cli/foo.rb` に置かれます。`new`・`import`・`doctor`・`upgrade`・`clean`・`create`・`delete`・`rename`・`lint`・`metrics`・`cover`・`resize` はこの形です。

規則から外れるものだけを挙げます。

コマンド	置き場所
`renumber`	`rename_command.rb` の `RenumberCommand`。実装は `cli/rename.rb` を共用
`index`・`index:plan`・`index:auto`・`index:apply`・`index:export`・`index:import`	`index_command.rb` ＋ `cli/index/` 配下
`pdf:read`・`pdf:compress`・`pdf:pages`・`pdf:rasterize`	`pdf_command.rb` ＋ `cli/pdf.rb`・`cli/pdf/` 配下
`open`	`open_command.rb` ＋ `cli/pdf.rb` の `PdfOpener`
`preflight`	`preflight_command.rb` ＋ `cli/build/pipeline.rb`
`build`	`build_command.rb` ＋ `cli/build/` 配下

`create:titlepage`・`create:colophon`・`create:legalpage` は、扉・権利表記・奥付だけを作り直すための内部コマンドです。ビルドはこれらのコマンドを経由せず、`CreateCommands` のメソッドを直接呼びます。`resize`（`images/` の PNG・JPG を WebP へ一括変換する）も内部コマンドです。ビルドが出力形式ごとに画像を用意するため、著者が素材を変換する必要はなく、`vs --help` とマニュアルからは外しています。

♣ ワークスペース

ビルドの中間生成物はすべて `.cache/vs/` の下に置かれ、原稿ディレクトリを汚しません。

```
.cache/vs/
  build/
    html/   pdf/   epub/   kindle/
    .volume
  derived/
    pdf/   svg/   image-metrics.yml
  math/   math-png/   mermaid/   showcase/   qr/   talk-avatars/
  covers/   theme-images/
  book-settings.css
  dedup-pages-<巻>.json
```

場所	内容
`build/html/`	前処理済み Markdown と変換後 HTML。すべての出力の原本
`build/pdf/`	PDF 用に加工したコピー。破壊的な書き換えはここに閉じる
`build/epub/` ・ `build/kindle/`	フレーバごとの消費者ディレクトリ
`build/.volume`	分冊の本で、最後に組んだ巻（巻の `id`・`combined`・`all`）。違う巻を組むときに置き場を空にする目印
`derived/`	ターゲットごとの画像派生。`pdf/` にラスタ、`svg/` に書体を埋めた SVG が入ります
`math/` 〜 `talk-avatars/`	生成資産のキャッシュ。final clean を生き延びます
`covers/` `theme-images/`	表紙・テーマ画像
`book-settings.css`	`book.yml` から生成した CSS
`dedup-pages-<巻>.json`	前回のビルドで取った索引・用語集のアンカーとページの対応。巻ごとに分かれ、巻のない本は `all`

生成資産のキャッシュは内容で引くので、ディレクトリができるのは実際に使った種別だけです。手元の `.cache/vs/` に上の全部が並んでいなくても、異常ではありません。

`html/` を原本とし、出力ごとにコピーへ加工するのが要点です。PDF 用の書き換えが EPUB へ漏れないのは、この構造そのものが保証しています。生成資産のキャッシュは通常のクリーンでは消えず、`vs clean --cache` が対象にします。

2 ビルドパイプライン

`vs build` は `Build::UnifiedBuildPipeline` が実行します。実装は `cli/build/pipeline.rb` です。

♣ 三つのモード

モード	起動	内容
full	`vs build`	全章。出力ターゲットに応じて全ステップを評価
single	`vs build 11-workflow`	単章。閲覧用 PDF のみ生成
preflight	`vs preflight`	原稿の検査のみ。HTML 変換・PDF 生成を行わない

単章ビルドが閲覧用 PDF だけを作るのは意図的です。入稿用 PDF・EPUB・Kindle は全章がそろってはじめて意味を持つ成果物なので、単章では中途半端な出力を避けています。

分冊の本では、どのモードでも、パイプラインに入る前に組む範囲（巻・合本・全巻）を一つ選びます（後述の「分冊」）。モードは範囲の選び方を変えません。

単章ビルドでは索引スキャンが走りません。索引語の初出を包む `<dfn>` が付かないため、索引・用語集に関わる見え方は全章ビルドでしか確認できません。

♣ 相と並列実行

full モードのステップは四つの**相**に属します。

```
:shared   ── 両方の出力が読む中間物を作る（HTML・目次・索引・表紙資産）
   │
   ├── :pdf     ── 閲覧用 PDF・入稿用 PDF の枝
   └── :epub    ── EPUB・Kindle の枝
   │
:join     ── 両枝の完了後。ワークスペースの掃除
```

`:pdf` と `:epub` は互いに独立なので、両方に仕事があるときは並列に走ります。EPUB 枝のログは合流時にまとめて出るため、分岐直後に予告を 1 行表示します。`VIVLIO_BUILD_PARALLEL=0` を与えると逐次実行に落とせます。並列を疑うときの切り分け手順は `build-pipeline-pitfalls-notes.md` にまとまっています。

♣ ステップ表

full モードのステップは、条件つきの 1 枚の表として宣言されています（`full_mode_step_table`）。**ステップ番号は持ちません。**実行条件によって並びが変わると番号は意味を失い、ドキュメントとログの表記がずれていくためです。代わりに**ラベル**が、ログ・計時・本章の共通語彙になっています。

相	ラベル	実行条件
shared	`clean`	常時
shared	`optimize images`	常時
shared	`prepare theme images`	常時
shared	`prepare cover assets`	表紙が要るとき
shared	`preprocess sections`	常時
shared	`index scan and build`	常時
shared	`convert sections html`	常時
shared	`generate part title pages`	常時

相	ラベル	実行条件
shared	`generate whole toc`	分冊の本で、`whole_toc: true` の巻を組むとき
shared	`generate front and back matter html`	常時
shared	`techbook post-process`	常時
shared	`generate toc html`	常時
pdf	`build overall pdf`	閲覧用 PDF が要るとき
pdf	`extract rotate table images`	回転テーブルがあるとき
pdf	`generate entries.js`	入稿用のみ・非導出のとき
pdf	`shrink oversized images`	PDF 系ターゲットのとき
pdf	`backlink dedup`	PDF 系ターゲットのとき
pdf	`build front and back matter`	閲覧用 PDF が要るとき
pdf	`merge all pdfs`	`pdf` ターゲット
pdf	`apply outline to output pdf`	`pdf` ターゲット
pdf	`shorten destination names`	`pdf` ターゲット
pdf	`compress, rename and final clean`	`pdf` 単独のとき
pdf	`compress and rename`	他ターゲット併存のとき
pdf	`print pdf`	`print_pdf` ターゲット
epub	`generate epub`	`epub` / `kindle` ターゲット
join	`final clean`	掃除を延期したとき

各行はもう一つ、**単位**を持ちます。`:chapter` は章ごとに意味を持つ処理、`:whole_book` は全章そろってはじめて成立する処理です。

単章ビルドはこの表から導出されます。 `register_single_mode_steps` が同じ表を走査し、`:chapter` の行だけを引いて単章の手順を組み立てます。モードごとの一覧は持たないので、全章に足したステップが単章へ届かない、という食い違いは起きません。単章で意図して走らせないステップは `SINGLE_MODE_SKIP`

に理由を添えて挙げます（いまは `techbook post-process` だけです）。詳しくは `build-mode-parity-spec.md`。

ステップを足すときは、この表に 1 行足して**相と単位を選びます**。分岐を書き足す必要はありません。単位の既定は `:whole_book` です——足し忘れたステップが単章へ流れ込まない側に倒してあります。

`vs build --log=debug` を実行すると、各ステップの開始・終了と所要時間が同じラベルで出力されます。

♣ 入稿用 PDF の導出

入稿用 PDF は、既定では閲覧用 PDF の中間成果物から導出されます（`derive_print?`）。本文を二度組まないぶん速く、閲覧用で確認した本文がそのまま入稿物になります。本文に紙端まで届く要素があり `full_bleed: true` を指定した場合のみ、塗り足し込みで個別にレンダリングします。

導出のときは `print_pdf` 単独指定でも閲覧用の中間 PDF を作ります。表の実行条件が `need_viewing_pdf` で連動しているのはこのためです。

♣ 本文ガード

本文 PDF を生成するステップには**本文ガード**が入っています。入稿用の本文はトンボ・塗り足し付きのもっとも重いレンダリングで、Chrome の一過性の失敗により本文が数ページに縮退することがありました。そこで生成後にページ数を検証し、本文相応に満たなければ再ビルドし、規定回数で回復しなければビルドを明示的に中断します。黙って本文の欠けた PDF を出荷しないための仕組みです。

♣ 前処理・変換・後処理

`:shared` 相の中身は、大きく三段に分かれます。

前処理（`cli/pre_process/`）は `MarkdownPreprocessor#run` が担い、フロントマター生成・データ展開・画像パス正規化・コードインクルード・コンテナ変換・数式や図の生成・リンクの脚注化などを順に適用します。原稿ファイルは一切変更せず、加工結果をワークスペースへ書き出します。

変換（`cli/convert.rb`）は VFM（Vivliostyle Flavored Markdown）を呼び出して Markdown を HTML にします。

後処理（`cli/post_process/`）は、生成された HTML に対して `<body>` へのクラス付与・組み込み置換・見出しのラップ・脚注のページ脚注化・行番号付与・見出し番号の付与などを適用します。

TIP

処理をどの段に置くかは「Markdown の構造を見たいか、HTML の構造を見たいか」で決まります。ブロックの切り出しやコード領域の保護が要るものは前処理、要素の入れ子や属性を触るものは後処理が向いています。

3 EPUB と Kindle の生成

`output.targets` に `epub` または `kindle` があると、`:epub` 相で `Build::EpubFlow` が動きます。

♣ フレーバ分離

`epub`（楽天 Kobo・Apple Books 向けのクリーン EPUB）と `kindle`（Amazon 向け）は、同じ `EpubBuilder` を**フレーバ**で切り替えて作ります。中心は `generate_epub_entries!(dir, entries, flavor:)` です。

- **共通: EPUB マーカー `vs-epub` の付与、索引・用語集の後処理、脚注 id の整理、テーブルの `align` 正規化、絵文字の復元、扉絵・節絵の注入**
- **共通（ローカライズ後）: 著者の SVG 図版を差し替え（クリーンは書体を抱かせたベクタのまま、Kindle はすべて PNG / JPEG へ焼く）**
- **Kindle 限定: WebP から JPEG/PNG への変換、`vs-kindle` マーカーの付与、画像サイズの制約、数式のテキスト化とラスター化、コードブロックの調整、囲みボックスへの実ラベル注入、異体字の 1 字書体（KFX は**

異体字セレクタを読まないので、禰 のような字は親字に異体の字形を割り当てた書体を埋め込んで出す）

`vs-epub` は「リフロー文脈である」ことを示す両フレーバ共通の印、`vs-kindle` は「Kindle 向けに劣化させた」ことを示す印です。**Kindle の `<body>` は両方を持ちます。**クリーン EPUB だけに当てたい CSS は `body.vs-epub:not(.vs-kindle)` と書く必要があります。

フレーバごとに消費者ディレクトリ（`epub/` と `kindle/`）が分かれているため、相互汚染は構造的に起こりません。表紙画像だけは両枝が同じ場所へ書くので、共通前段の `prepare_cover_assets` が一度だけ作ります。

♣ KPF 変換

`kindle` ターゲットでは、中間 EPUB を作ったあと `convert_epub_to_kpf!` が `kindlepreviewer`（Kindle Previewer 同梱の CLI）を呼んで `.kpf` を生成します。未インストールなら中間 EPUB を残して変換だけスキップし、警告に留めます。Kindle を使わない利用者のビルドを止めないためです。変換ログのエラー・警告コードは集計して要約表示します。

Kindle Previewer 4（4.0.3）は、macOS でときどき変換の入口で落ちます（変換の本体を起動する `fork` の直後に子プロセスが止まります）。このときは本の変換ログ（`Logs/*_log.csv`）が残らないので、それを目印に 1 回だけやり直し、2 回とも落ちたらクラッシュレポートの場所を添えて再ビルドを案内します。

呼び出しのオプションは版で綴りが違います。3 は `-convert`、4 は `--convert` です。実行ファイルの名前と置き場所も変わっているので、`kpf_option_prefix` が `-help` の出力を一度だけ読んで綴りを決め、`vs doctor` が新しい版を先に探します。版を決め打ちにすると、どちらかの利用者の手元で KPF 変換だけが黙って失敗します。

♣ Kindle に SVG を残さない

Kindle Previewer 4 は、同梱された SVG があると Enhanced Typesetting を無効にします。 本文がその SVG を参照しているかどうかは関係がなく、

`content.opf` の manifest に載っているだけで判定が変わります。無効と判定された本は `.kpf` ではなく `.mobi` になるので、KDP へ入稿するものが作れません。

この判定は 4 で入りました。同じ EPUB を通すと、3 は `.kpf` を、4 は `.mobi` を作ります。

対処は 2 段です。

1. **参照されている SVG は焼きます。 `stage_author_svg_for_epub!` が、書体を抱かせた派生を作れない図——ロゴや QR のように文字を持たないもの——も、Kindle に限ってラスタ化します**
2. **参照の切れた SVG は外します。 `sweep_unreferenced_assets!` が、ローカライズで運ばれたのに誰も見ていない画像をパッケージから落とします。SVG もその中に入ります**

2 が要るのは数式のためです。ディスプレイ数式は PNG へ、インライン数式はテキストへ移ったあと、元の SVG だけがパッケージに残ります。

この回収はクリーン EPUB にも掛けます。ローカライズは `images/` と `stylesheets/` を参照を見ずに丸ごと運ぶので、原稿で使っていない雛形の画像（扉絵の花やサンプル章の写真）まで入るためです。照合はファイル名で行い、表紙（`covers/`）は対象から外します。

呼ぶ位置が効きます。 回収は参照の有無で判断するので、HTML の書き換えがすべて終わった後——ローカライズよりも、図版の差し替えよりも後——でなければなりません。早すぎると、まだ使われている SVG を落とします。

> **空のディレクトリを残すと、変換そのものが失敗します。** SVG を外した跡にディレクトリだけが残ると、Kindle Previewer 4 は 1 秒で `Book Conversion failed` を返し、ログも残しません。epubcheck は `PKG-014` の**警告**で通してしまうので、検証を足しても気づけません。`prune_empty_dirs!` が、変化がなくなるまで畳みます。

画質は落ちません。Previewer 3 が作っていた KPF も、中身のリソースはすべて JPEG でした。詳しい実測は `kindle-css-compatibility-notes.md` §5.5 にあります。

♣ KFX の CSS 制約

Kindle の表示エンジン（KFX）は、EPUB で広く使える CSS の一部を解しません。`body.vs-kindle` 配下のフォールバックを書くときは、以下を避けて具体値で記述します。

- **`:is()` セレクタ（ルールごと破棄される。複合セレクタは展開して書く）**
- **`var()` / `calc()` / `clamp()` などの CSS 関数**
- **`display: grid`、`linear-gradient()`、`::before` の絶対配置**
- **WebP 画像**
- **改ページは `break-before: page` より旧来の `page-break-before: always` が確実**

対応状況の一覧と回避策は `kindle-css-compatibility-notes.md` が正典です。

4 主要サブシステム

♣ 分冊

`catalog.yml` に `VOLUMES:` を書くと、一つの `contents/` から上下巻などを組めます。中心は `Build::Volumes`（`cli/build/volumes.rb`）で、巻は順番のある並びとして扱います。巻の数も名前の意味も決め打ちせず、「前の巻」「最初の巻」「最後の巻」は並びの順から決めます。

組む範囲は、プロセスに一つだけ持ちます。 コマンドの始まりに `Volumes.select!` で選び、以後は `Volumes.selection` を誰もが読みます。

選択	起動	目次（CatalogLoader.load_catalog が返すもの）
巻（Volume）	vs build --volume lower・章を指定した vs build 21	その巻の目次
:combined	巻を選ばない vs build（合本）	最初の巻の前書き・全巻の本文と付録・最後の巻の後書き
:all	既定。巻を選ばない vs preflight・vs lint・vs metrics	どれかの巻に載った原稿すべて

load_catalog は、どの選択でも巻のない本と同じ形の Hash（PREFACE・CHAPTERS・APPENDICES・POSTFACE）を返します（Volumes.view）。部の中扉、最初の本文の章を探す処理、索引の対象など、目次を読むほかの処理は巻を知らずに済みます。**目次を読む処理を足すときは load_catalog を通してください。** load_raw_catalog を直接読むと VOLUMES: の入れ子が見え、巻を選んでも全巻を読んでしまいます。一時的に別の範囲で数えたいとき（全巻の原稿で用語集の警告を出す、など）は Volumes.with_selection(:all) { … } で包みます。

--volume all は、巻ごとに別のプロセスで組みます。 CONFIG も章の並びの記録もプロセスの中に残るので、同じプロセスで巻を切り替えると前の巻の値が混ざるためです。BuildCommand#run_all_volumes が巻の順に vs build --volume <id> を起動し、失敗した巻で止めます。vs cover --volume all だけは同じプロセスで巻ごとに選び直します。表紙が読むのは CONFIG だけなので、作り終えたら元の CONFIG を入れ直せば足ります。

巻を選ぶと、ほかに三つのことが起こります。

- **書誌情報の上書き: 巻に書いた book: のキーを book.yml の値に重ねた CONFIG を入れます（Volumes.apply_config!）。扉・奥付・表紙・PDF の文書情報・EPUB の書誌情報はどれも CONFIG を読むので、そのまま巻の値になります。上書きできるキー（BOOK_KEYS）は ConfigKeys.book_keys から作るので、book: にキーを足せば巻でも上書きできます**

- **置き場の掃除: 前に別の巻を組んだ HTML が残ると、本文の章の並びに混ざって番号がずれます。`Volumes.prepare_workspace!` が `build/.volume` と比べ、違えば `--no-clean` でも置き場を空にします**
- **出力の名前: `project.name` の後に巻の `id` が入ります（`mybook_lower_v1.0.0.pdf`）。ISBN のない EPUB の識別子も、`project.name` と `id` から作ります。同じ識別子だと、ストアや読書アプリが上巻と下巻を同じ本の版違いとして扱うためです**

番号は `Volumes` だけが数えます。

番号	決め方	実装
本文の章	前の巻から続ける。`chapter_start` を書いた巻はその番号から	`chapter_starts`・`chapter_offset`
付録の記号	全巻を巻の順に見て、はじめて出た付録に次の記号。再び載る巻でも同じ記号。`appendix_start` を書いた巻はその巻の付録だけを振り直す	`appendix_letter`
ノンブル	巻ごとに 1 から	（巻のない本と同じ）

章扉の「第 N 章」は、全章ビルドで巻を選んだときだけ `BuildCommand` が `HeadingProcessor.chapter_number_offset` に足す数を渡します。単章ビルドの章扉は、巻を選んでも「第 1 章」から数えます（`chapter-reference-spec.md` の決まり）。目次の番号は CSS のカウンタで数えているので、HTML の章扉に書いた番号は届きません。`TocGenerator` が目次の最上位の `<ul>` と付録の `<li>` に `counter-reset` を与えて合わせます。

本文の章や付録は、二つ以上の巻に載せられます。 番号は載った巻ごとに、巻の中の位置で振ります。合本と `:all` の目次には、最初に載った所に一度だけ残します（`merged_sections`）。原稿の名前で見るので、部の中に書いた章も部ごとに別物とはみなしません。巻をまたいで並びを見る検査（「前の章」）は、共通の章が巻ごとに現れる `reading_order` を使います。

巻をまたぐ参照は、ラベルを全巻の原稿から集めて作ります（`CrossReferenceProcessor`）。組む範囲に入らない原稿

（PreProcessCommands.other_volume_files）のラベルは、その巻を選んだ状態（with_selection）で数えるので、番号はその巻の振り方になります。別の巻への参照は にし、リンクにしません。@pageref もページ番号を付けず、章で示します。ほかの巻はその場で組まないので、ページ番号が分からないためです。

全巻目次は Build::WholeTocGenerator が作ります。扉や中扉と同じシステムページ（_whole_toc）で、その巻の目次の前に置きます。ほかの巻の HTML はないので、節は原稿の Markdown の ## の行から取ります。題名は @chapref と同じ plain_chapter_title、節の番号は章扉と同じ材料（heading_number_context_for）で作ります。今の巻には vs-current-volume を付けて濃く組みます。Kindle は var( ) を読まないので、色はリテラルで書きます。ページ番号とリンクは載せません。

巻の書き方の誤り（id の重なり、知らないキー、ISBN の共有など）は Guards::VolumeDefinitionCheck が知らせ、vs build と vs preflight で同じものを出します。vs create・vs delete・vs rename は CatalogUpdater を通して、すべての巻の行を書き換えます。設計の全体と、まだ作っていないもの（ほかの巻のページ番号が要る索引・全巻目次のページ番号・全巻を通したノンブル・背表紙）は multivolume-spec.md と PLANNED.md にあります。

♣ 索引・用語集

Vivliostyle の target-counter はレンダリング時にページ番号を解決するため、Ruby のビルド時点では実際のページ番号が分かりません。そこで「**Ruby がリンク構造を作り、CSS がページ番号を流し込む**」設計を採っています。

本文中の索引語は <dfn id="idx-..."> （初出）または （二度目以降）に変換され、索引ページの <a href="chapter.html#idx-..."> が target-counter(attr(href), page) でページ番号を描きます。

索引と用語集は config/index_glossary_terms.yml の単一ファイルで管理し、各語の flags で所属を決めます。

flags	掲載先	本文への挿入
`i`	索引のみ	`<span class="index-term">`
`g`	用語集のみ	`<span>` ＋ `<a class="glossary-link">†</a>`
`ig`	両方	`<span>` ＋ `<a class="glossary-link">†</a>`

索引の主要参照（その語を説明している章）は太字で先頭に置き、副次参照を後ろに続けます。同じページを指す重複は `backlink dedup` が排除します。組み上がった PDF の named destinations を読んで「アンカー ID からページ番号」を得るので、章が複数ページにまたがっても正確です。

重複を除くと索引が縮んでページが動くので、除いた HTML で本文をもう一度組みます。本書ではこの 2 回目が約 55 秒かかります。これを省くために、**前回のページ対応で先回り**します（`BacklinkDedupOrchestrator`）。

1. **`backlink dedup` は、取ったページ対応を `dedup-pages-<巻>.json` に残します**
2. **次のビルドでは、`build overall pdf` が HTML を `pdf/` へ移した直後（`speculate!`）に、前回の対応で索引・用語集の重複を先に除いてから 1 回目を組みます**
3. **`backlink dedup` は、先回り前の HTML に今回の対応と前回の対応をそれぞれ当て、結果の文字列が同じなら 2 回目を組みません。違えば今回の対応で組み直すので、出来上がりは先回りしないときと変わりません**

ページ対応そのものではなく当てた結果を比べるのは、判定に効かないずれ（章は改ページで始まるので、ずれは直した章の中で吸収されやすい）を外れに数えないためです。縮小版の画像へ差し替えたとき（`shrink oversized images`）と、本文の † を除いたときは、比べずに組み直します。

> **先回りで消したリンクは、消さずに残してください。** 消したリンクの飛び先は PDF の `/Dests` から消えるので、今回の対応が取れず、何も直さなくても外れ続けます。消したリンクは、大きさ 0 のリンクとして索引・用語集の末尾（`div.vs-dedup-witness`。`<dd>` の外）に残します。範囲表記（12–15）で間を詰めたリンクも同じです。

索引語のアンカーは語ごとの通し番号（idx-<語のダイジェスト>-N）なので、本文で索引語を書き足す・消すと、それより後ろの同じ語の番号がずれて外れます。--log の外れた理由は、ページのずれと参照の増減とを分けて出します。経緯と実測は dedup-rerender-skip-spec.md にあります。

著者向けの記法と運用は、本書の「索引・用語集機能」の章にあります。

索引語は本文に埋め込まれる透明なアンカーです。初出を包む <dfn> はブラウザ既定で斜体になるため、打ち消しを stylesheets/base.css に置いています。索引語が現れるのは本文・見出し・目次のすべてなので、全ページが読む CSS でなければ届きません。

♣ 数式

数式は前処理で SVG に組まれ、Kindle だけが後段でテキストへ落とされます。四つの部品が順に並びます。

```
MathSpanDetector      どのバッククォート・フェンスが数式かを決め、
   │                  数式デリミタへ起こす
   ↓
PlainMathTranspiler   素の表記（√n・x²・a^(p-1) mod p）を TeX へ変換する
   │
   ↓
MathTransformer       MathJax を呼んで SVG を生成し、キャッシュに載せる
   │
   ↓
MathTextRenderer      Kindle 向けに TeX を HTML のテキストへ組み直す
```

前の三つは cli/pre_process/、MathTextRenderer は cli/build/ にあります。SVG の生成は mathjax_to_svg.mjs を呼び、結果は GeneratedAssetCache に載ります。

原稿に LaTeX は書かせません。 TeX はビルド中だけの中間表現で、著者が読み書きするのは常に素の表記です。a^{p-1} の a を b に直す、という普通の編集がそのまま通ります。判定を通したくないときは ```math フェンスで宣言できます。

判定は「コードのしるしがあればコード」から始めます。 数式のしるしを先に見ると `array[i] * 2` を拾ってしまうためで、拒否を優先したうえで数式のしるしを探します。日本語を含む綴りは、`^` や √ のような強いしるしがあるときだけ数式にします。

> **MathJax は素の表記をエラーにしないまま壊します。** `x²+y²` は上付きが落ちて `x+y` に、`H₂O` は `HO` に、`a・b` は中黒が消えて `ab` になります。ビルドは成功し警告も出ないため、著者は刷り上がりを見るまで気づけません。変換器はこの無言の欠落を塞ぐためのもので、親切機能ではありません。

Kindle は画像を本文の文字サイズへ追従させる手段を持たないため、**インライン数式を三段に落とします**。`MathTextRenderer` が HTML のテキストへ組めればそれを使い、組めなければ著者が書いた原文テキストを、それも無ければ SVG を残します。二段目があるおかげで、素の表記で書かれた式はテキスト化に失敗しても追従します。

ディスプレイ数式は Kindle でも画像のままです。総和や積分を独立行に組む意図を保つためで、行を占有するので崩れません。こちらは `rsvg-convert` で PNG へ焼きます。

日本語を含む数式の SVG には `SvgFontEmbedder` が**本文書体**をサブセットで埋め込みます。`<img>` から参照される SVG は独立した文書で `@font-face` が届かず、埋めないと OS のフォントへ落ちて PDF に Type 3 が混入します。見出し書体ではなく本文書体を使うのは、数式の中の日本語が本文の続きだからです。

詳細は `plain-math-notation-spec.md` にあります。

♣ 表紙

表紙は `CoverCommands` が作ります（`cli/cover.rb`）。入口は `vs cover` と `vs build` の二つで、どちらも同じ `generate!` を通ります。判型は `page.use`、作るものは `output.targets` で決まります。

表と裏は、それぞれ元の画像を決めます（`resolve_source`）。

1. **`covers/<side>cover_<theme>.png` があれば PNG の経路（`cover.rb`）**
2. **無ければ著者の `covers/<side>cover_<theme>.svg`、それも無ければ同梱の `covers/bundled/<side>cover.svg` を SVG の経路（`cover/svg_cover.rb` の `SvgCover`）。同梱テンプレートにはパレットと書誌を、著者の SVG には書誌だけを当てます**

ターゲット	生成物	直下への複製
`pdf`	`<side>cover_<theme>_<size>_rgb.pdf`（仕上がりサイズ）	しない（閲覧用 PDF に結合）
`print_pdf`	`<side>cover_<theme>_<size>_cmyk.pdf`（塗り足し・トンボ・PDF/X-1a）	する（入稿用）
`epub` / `kindle`	`cover_<theme>.jpg`（表のみ・1,600 × 2,560 px）	する（KDP 用）

入稿用の表紙のトンボは、本文の入稿用 PDF と同じ部品（`CropMarksOverlay` と qpdf の overlay）で重ねます。仕上がりと塗り足しの範囲（`TrimBox`・`BleedBox`）は、トンボを重ねたあとで書きます。先に書くと、qpdf がトンボを仕上がりの大きさへ縮めて置くためです。塗り足しの幅も、本文と同じ設定から同じ読み方で読みます。

♣ ターゲット別の画像派生

著者が置いた `images/` の画像には触れません。 ターゲットごとに適した形式・解像度の派生を `.cache/vs/derived/` に作り、そちらを組版へ渡します。実装は `Build::DerivedImage`（`cli/build/derived_image.rb`）です。

ターゲット	ラスタ画像に渡す形式	理由
PDF	JPEG（透過が要れば PNG）	WebP を格納できない
EPUB	素材のまま（WebP）	リーダーが WebP を解する
Kindle	JPEG / PNG	KFX が WebP を解さない

PDF が WebP を格納できないのは、ISO 32000 に該当するフィルタがないためです。渡すと Chromium がデコードして Flate へ入れ直します。可逆圧縮では写真がほとんど縮まないため、**素材 280 KB が PDF 内で 1,963 KB になります。**

JPEG なら DCTDecode でストリームがそのまま入り、素材のサイズがそのまま PDF のサイズになります。

解像度も版面基準で詰めます。shrink oversized images が組み上がった PDF から実効解像度を測り、過剰な画像を縮小版へ差し替えます。測定結果（画素数と「版面に対する表示幅の割合」の対応）は .cache/vs/derived/image-metrics.yml にビルドをまたいで残します。

> **TIP**
>
> **割合を残すのは EPUB / Kindle のためです。** リフローではページが確定せず「組んでから測る」ができませんが、版面に対する相対幅は組版系によりません。width=30% の指定も 2 列に並べた表も PDF 側の測定にすでに現れているので、その割合を読者の画面幅へ掛ければ必要画素数が出ます。

実測では PDF が 97.2 MB → 27.3 MB、EPUB が 30.5 MB → 18.1 MB になりました。効いたのは縮小より**形式の変換と透過のフラット化**です（透過を抱えたままでは可逆圧縮が効きません）。詳細は image-format-per-target-spec.md。

♣ SVG 図版の派生

ラスタ画像とは別の経路があります。**著者が images/ に置いた SVG は、書体を抱かせた複製へ差し替えて渡します。** 実装は Build::DerivedSvg（cli/build/derived_svg.rb）です。派生は .cache/vs/derived/svg/ に置きます。

<img> から参照される SVG は独立した文書です。本文 HTML の @font-face と CSS 変数は届かず、相対パスの外部フォントも読めません。font-family に何を書いても解決されないので、OS 既定のフォントへ落ちます。PDF ではそれが Type 3 フォントとして埋め込まれ、入稿できなくなります。

DerivedSvg は図に出る字だけを抜き出したサブセットを作り、data: URI の @font-face として SVG 自身へ持たせます。8 文字で 3.3 KB 程度なので、図は実質太りません。図が実体のある書体を名指ししていればそのまま、汎用名しか

無ければ書籍の書体をスタックの先頭へ足します（`sans-serif` は見出し、`serif` は本文、`monospace` はコード書体）。**Regular と Bold を別々に埋めます**——1 面だけだと図が太字を要求したときに疑似太字が合成され、それがまた Type 3 になるためです。

ターゲット	渡すもの	書体
PDF	派生 SVG（ベクタ）	書籍の書体
EPUB	同じ派生 SVG	書籍の書体
Kindle	PNG / JPEG	組んだ機械の OS 書体

EPUB にも同じ派生を配ります。 ラスタライズで逃げる手は使えません。`rsvg-convert` は `data:` URI の `@font-face` を読みません。macOS では `fontconfig` に登録した書体も見えません。焼けば**そのとき使った機械の既定フォント**が図に焼き付きます。別の機械で組めば別の書体になり、成果物が再現しません。ベクタのまま運べば決定論で、しかも軽く、高精細な画面でも鮮明です。

Kindle だけは KFX が SVG を扱えないので、EPUB 枝の `stage_author_svg_for_epub!` が焼きます。透過があれば PNG、無ければ JPEG です。ここで書体が OS 依存になるのは避けられません（直すには Chromium で焼く必要があります）。

文字を持たない図（ロゴのアウトライン、QR コードなど）と、すでに書体を抱えている生成 SVG（`showcase`・`mermaid`・数式）は対象外で、素材のまま運ばれます。経緯と実測は `type3-font-embedding-notes.md` §9。

♣ 日本語校正

`vs lint` は、textlint の検査・Ruby 側の独自ルール・英語のスペルチェックを、ひとまとめに走らせるコマンドです。実装は `cli/lint/` にあり、textlint では扱えない判定を Ruby 側が受け持ちます。

独自ルールは `ProseChecker` に 17 個あります。形態素の境界を見るもの、Markdown の記法を見るもの、原稿全体の状態を見るもの——いずれも textlint の枠組みでは書けないものです。

種類	ルール
日本語の読み違い	mazegaki・ambiguous-comparison・missing-period・ja-no-weak-phrase
長さ	sentence-length・long-parenthetical
表記	kanji-lookalike・kansuji-counter-suffix・slash-between-japanese・space-around-brackets・index-term-spacing・affix-spacing・hiragana-spacing
記法の取り違え	indented-code-block・setext-heading・multiple-chapter-headings
抑止の閉じ忘れ	unclosed-suppression

`--fix` で直せるのは mazegaki・kanji-lookalike・index-term-spacing・affix-spacing・hiragana-spacing の五つだけです。「この字はこう書く」が一つに決まるものしか入れていません。残りは直し方が文脈で変わるので、著者に委ねます。ルール名は著者が `book.yml` の `lint.disabled_rules` に書く名前でもあるので、上流のルールと同じ名前空間に置いています。

自前のルールが上流と重なったら、上流を切ります。 どれを切るかは `SUPERSEDED_TEXTLINT_RULES` の 1 表が持ちます。

プリセット	切るルール
preset-japanese	no-kanji-lookalikes・sentence-length
preset-ja-technical-writing	ja-no-mixed-period・sentence-length・ja-no-weak-phrase
preset-ja-spacing	ja-no-space-around-slash・ja-no-space-around-parentheses

切らないと、同じ箇所に 2 件の指摘が並びます。置き換えた理由はルールごとに違います。たとえば ja-no-weak-phrase は、MeCab が するかもしれません を か ＋ も に切ってしまうため上流では拾えず、独自ルールで受け持っています。**ルールを足すときは、この表も見てください。**

交ぜ書きの辞書は 2 段構成です。

段	語数	判定	必要なもの
第 1 層	約 150	綴りの一致だけで決まる	なし
第 2 層	約 1,900	形態素の境界を見て決まる	MeCab

分けてあるのは、綴りだけでは外せない誤検出があるためです。**う回**（迂回）は「使う回数」「という回答」に紛れ、前後のどの文字を除いても区別できません。MeCab に読ませると「使う｜回数」と語が切れるので、そこではじめて外せます。

> **同じ原稿でも環境によって指摘数が変わります。** MeCab のない環境では第 2 層が丸ごと黙ります。ただし**減るのは指摘だけで、誤った指摘が出ることはありません**——判定できないものを黙って通す側に倒してあります。回帰テストを書くときは、この非対称を前提にしてください。

判定の入力は**読者が見る文字列**でなければなりません。結**合し**直した のように語の途中へ強調記法が入ると、素の行を見る実装では前後を制約するガードが * を見てすり抜け、逆に あいさつ**文** のような語は分断されて取りこぼします。`Masking.strip_emphasis` で記法を外した文字列で判定し、位置を元の行へ戻します。

指摘が誤っているとき、直す場所は三つあります。 どれを選ぶかで、影響の及ぶ範囲が変わります。

直す場所	何を直すとき
`config/textlint_dictionaries/` の辞書	辞書の項目そのものが誤っているとき。まずここを疑ってください
`BUILTIN_ALLOWLIST`（`cli/lint.rb`）	上流のルールの取りこぼしを、ツール側で打ち消すとき
`config/textlint_allowlist.yml`	著者が「この本ではこう書く」と決めるとき

`BUILTIN_ALLOWLIST` は、実行時に組み立てる `.textlintrc.yml` の `filters.allowlist.allow` へ注ぎます。フィルタを通るので **`--fix` にも効きます**——textlint は、フィルタで消えた指摘には修正を当てません。上流の不備を著者の `textlint_allowlist.yml` へ書かせると、新しい本を作るたびに同じ 1

行を書き写すことになります。ツール側の打ち消しは、ツール側に置いてください。

記法の綴りは NotationGuard だけが持ちます。 `cli/lint/notation_guard.rb` は、lint が見る記法を一箇所へ集めた場所で、二つを預かります。

- **数式の退避** — **`$…$` を目印へ置き換えてから校正へ渡し、位置を元の行へ戻します**
- **抑止コメントの綴り** — **`<!-- no-lint -->`・`<!-- no-lint-start -->`・`<!-- no-lint-end -->` と、廃止した旧記法（`vs-lint-*`）から置き換え先を案内するための対応表**

> **この判断を写して持たせないでください。** 抑止コメントの照合は `prose_checker.rb`・`lint.rb`・`tokenizer.rb` の 3 箇所が使いますが、どれも `NotationGuard` の定数を参照します。写しを持つと「行全体がコメントのときだけ抑止とみなす」という判断がずれ、抑止記法を解説した文そのものが lint を止めてしまいます。

記法の知識を `Masking` へ置いてはいけません。 あちらは索引・メトリクス・前処理など 20 箇所以上が通る共通基盤なので、記法の知識を混ぜると lint と無関係な処理まで意味が変わります。

♣ メトリクス

`vs metrics` は章ごとに解析し、結果を `.cache/metrics/{basename}.yml` へ残します。**鮮度は章ファイル自身の mtime と比べ、章ごとに独立して判定します。** 1 章だけ直したときに解析し直すのは、その章だけです。

キャッシュに入れるのは表示する値ではなく、**組み立て直せるだけの素の値**です。

キー	中身
`basic_stats`	文字数・行数・文・節の統計

キー	中身
`vocabulary_stats`	`tokens_map`（語ごとの出現数）・MATTR・漢字文字数・総語長
`readability`	スコアとラベル、全体集計用の特徴量（文・連・句読点の数）
`sections`	節ごとの文字数と警告

特徴量まで持つのは、**全体値が章ごとの値の平均ではない**ためです。読解難度（RS）は各章の連や文の数を合算したうえで一度だけ算出し、語彙多様度（MATTR）は語数で加重平均します。素の値さえ残っていれば、全体集計も `--json` / `--yaml` も再解析なしで組み立てられます。常にフル解析させたいときは `VIVLIO_METRICS_CACHE=0` を設定します。

章の解析は並列で走ります。既定の並列度は **CPU コア数と 4 の小さいほう**で、`VIVLIO_METRICS_CONCURRENCY` で上書きできます。**表示は章番号順に整列する**ので、解析の終わった順に出ることはありません。

画面への出力は次の順で進みます。

1. **章別リストの逐次出力: 章の解析は並列でも、表示は章番号順です。先に終わった章があっても前の章を追い越しません。`--sections`・`--warn`・章指定の絞り込みはこの段階で当て、対象が 1 章もなければ「対象章がありません」と表示します。**
2. **全体集計: 全章の解析が終わってから、基本情報・文構造・詳細分析をまとめて表示します。**
3. **推敲用の参考資料: `--all` のときだけ、章間のばらつき以降を続けて出します。**

棒グラフの縮尺は、解析の前に文字数を概算して先に決めます。章が出るたびに横幅が変わることはありません。

`--json` / `--yaml` は逐次出力を省き、次の形で一括して返します。

```
{
  "stats": [
    { "path": "contents/11-intro.md", "chars": 1200, "warnings": [], ... }
  ],
  "totals": {
```

```
    "chars": 2400,
    "avg_sentence_chars": 210.5,
    "vocabulary": { "mattr": 0.586, "kanji_ratio": 27.1, ... },
    "readability": { "score": 42.1, "label": "Standard" }
  },
  "advice": {
    "consistency": [ { "label": "漢字比率", "mean": 27.38, "high": [...], "low": [...] } ],
    "long_sentences": [ { "chapter_num": 3, "line": 274, "length": 118, "text": "…" } ],
    "sentence_rhythm": { "distribution": {...}, "monotone_runs": [...] },
    "content_words": [ { "word": "画像", "pos": "名詞", "count": 306 } ],
    "kanji_levels": { "ratios": [...], "lists": {...}, "locations": [...] }
  }
}
```

`totals` は上で述べた素の値から組み立てた全体値なので、外部ツールで平均を計算し直す必要はありません。`advice` は画面の `--all`（推敲用の参考資料）を構造化したものです。どちらもキャッシュから組み立てるので、章を再解析せずに返せます。

♣ QueryStream

`data/*.yml` のデータと `templates/_book.md` などのテンプレートを組み合わせ、原稿の 1 行を一覧へ展開する機能です。実装は `query-stream` gem として独立しており、`cli/pre_process/data_render.rb` が呼び出します。

`QueryStreamParser` が `= books ¦ tags=ruby ¦ :full` をパースし、`DataResolver` がデータファイルを探し（`book` と書いても複数形に補完します）、`FilterEngine` が絞り込みと並べ替えを行い、`TemplateCompiler` がテンプレートへ流し込みます。

テンプレートは `templates/_<単数形>.<スタイル>.md` の規約で解決されるため、`_book.mystyle.md` を置くだけで `= books ¦ :mystyle` が有効になります。`data/elements.yml` と `templates/_element.md` を置けば、設定を変え

ずに = elements が使えます。新しいデータ種別を足すのに、コードを書く必要はありません。

記法の詳しい説明は、本書の「データ展開機能」の章にあります。

♣ CrossReference

図・表・コードリストに「章番号＋連番」を自動付与し、本文から参照リンクを張る機能です。実装は cli/pre_process/cross_reference_processor.rb で、前処理の中で全章を一括して処理します。

1. **ラベルの収集 — LabelCollectorContext が全章を走査してラベルマップを構築。重複 ID はエラーとして記録**
2. **ブロックの変換 — CaptionedBlockTransformer がキャプション行と直後のブロックを HTML にし、id と章番号付きの連番を埋め込む**
3. **参照の置換 — ReferenceReplacer が本文の参照をラベルマップと照合し、番号付きリンクへ置き換え**

全章のラベルを集めてから置換する必要があるため、この処理だけは章ごとではなく一括で走ります。

見出しに付けたラベルのアンカーは、見出しの**内側**に空要素として置きます。外に置くと break-before: page でアンカーだけが前ページに落ち、参照先のページ番号が一つずれるためです。

♣ @ ディレクティブの二トラック

@ で始まる記法は、実装される場所が二つに分かれています。

記法	処理する場所	実装
@pageref:id ／ 見出しラベル	前処理（全章一括）	CrossReferenceProcessor
@qr:URL	前処理（章ごと・画像生成を伴う）	QrTransformer
@vspace @hspace @pagebreak @version @title @today	後処理（HTML への文字列置換）	ReplacementRules

後処理の置換は PDF・EPUB・Kindle が共有する HTML に一度だけ効きます。**出力ごとの差は Ruby ではなく CSS のカスケードで吸収します。** `target-counter()` や `break-before: recto` を解さないリーダーは、その宣言だけを捨ててフォールバックを採るためです。

`@version` / `@title` / `@today` は値が `book.yml` とビルド時刻に依存するため、凍結定数ではなく適用のたびに組み立てられます。定数にすると設定を読み直したあとも古い値を差し込んでしまいます。

♣ PDF プロバイダの二系統

PDF の低水準の操作のうち、ページ数の取得、白紙ページの作成、PDF アウトライン（しおり）の付与の三つは、`Pdf.provider` が選ぶ実装に委ねます（`cli/pdf/provider.rb`）。

プロバイダ	ライセンス	機能
Standard	MIT（Prawn・pdf-reader）	ページ数・白紙ページ（しおりは付けず警告する）
Enhanced	AGPL-3.0（HexaPDF）	上の二つ＋しおり

AGPL のライブラリを本体から切り離すための構造です。Standard（`StandardProvider`）は本体に、Enhanced（`EnhancedProvider`）は別 gem の `vivlio-starter-pdf` にあり、この gem を入れてあれば自動で Enhanced が選ばれます。`VIVLIO_PDF_PLUGIN=disable` を与えると、常に Standard になります。`vs pdf:read` の画像抽出・OCR も同じ切り分けです。

隠しノンブルはプロバイダを通りません。`NombreStamper` が Prawn で描き、qpdf の overlay で重ねます。どちらのモードでも同じ処理です。**プロバイダへ回す処理は増やさないでください。** 回した処理は、プラグインの有無で振る舞いが変わります。

> プラグインを入れた開発機ではビルドが Enhanced 経路を通るので、Standard だけを直しても実ビルドの挙動は変わりません。両方を直し、プラグイン側は `gem build` とバージョン更新を経て入れ直してください。

5 拡張の手引き

♣ 著者向けの記法を足す

`:::{.class}` の囲み、```` ```mermaid ```` のようなフェンス、インライン記法、生成資産（SVG・画像）を新設するときは `notation-implementation-guide.md` に手順があります。最初に読むべきは §2 の**再実装禁止リスト**です。

基盤	責務
`Masking`	コード領域解釈の唯一の実装。フェンスとインラインコードの解釈は必ずここを通す
`IndexMarkup`	索引マークアップ `[用語]` の綴りの唯一の定義。`[…]` と綴る記法を足したら除外を 1 項目足す
`MarkdownUtils`	`extract_code_spans` と `restore_code_spans`。コードを退避して処理し復元する標準手順
`GeneratedAssetCache`	生成資産の永続キャッシュ
`MarkdownTransformer`	`convert_container_blocks` が `:::{.class}` をタグへ変換
`ToolUpgrader::TOOLS`	外部ツールの正典。`doctor --fix` と `vs upgrade` が共用
`Common.book_language`	本の言語の唯一の解決規則。`<html lang>`・`dc:language`・kindlepreviewer の `-locale` が共用
`Common.external_command_available?`	外部コマンドの有無の確かめ方。`which` を起動せず PATH を見る。Windows の PATHEXT もここで扱う
`Common.image_size`	画像の寸法（`magick identify`）の読み方。読めなければ `nil`

これらと同じ責務のコードを書き始めたら、それは基盤側に API を足すサインです。

囲みボックスを新設するときは Kindle 劣化の三点セットを併せて行います。`EpubBuilder::ADMONITION_LABELS` へラベルを足し、`chapter-common.css` に `body.vs-kindle` 配下の規則を**リテラル色**で書き、レイアウトテストにラベルの検査を足します。KFX は `::before` と `var()` を無視するため、この三つがそろってはじめて Kindle でも囲みとして読めます。

♣ ビルドステップを足す

`full_mode_step_table` に `[ラベル, ハンドラ, 実行条件, 相, 単位]` を 1 行足します。単位は前述のとおり `:chapter` か `:whole_book` で、相の選び方が設計判断の中心です。

- **両方の出力が読む中間物を作るなら `:shared`**
- **PDF だけが要るなら `:pdf`、EPUB・Kindle だけなら `:epub`**
- **両枝の完了を待つ必要があるなら `:join`**

`:shared` に置いたステップは並列化の恩恵を受けませんが、`:pdf` と `:epub` に同じ処理を二度書くよりは安全です。逆に、両枝が同じファイルへ書くステップを枝側に置くと競合します（表紙資産を `:shared` に置いているのはこのためです）。

♣ 設定キーを足す・やめる

`book.yml` のキーと既定値は `cli/config_keys.rb` の 1 表が正典です。ただし手を動かす前に `config-key-criteria-guidelines.md` の五つの判断軸を読んでください。「そのキーは設定であるべきか」を問うもので、足す手順より先に立ちます。足す手順は `config-extension-guidelines.md`、やめる手順は `config-retirement-guidelines.md` にあります。

`book:` の下に足したキーは、分冊の本では巻ごとに上書きできるようになります（`Volumes::BOOK_KEYS` が `ConfigKeys.book_keys` から作られるため）。巻ごとに変えては困る値なら、`book:` の外に置いてください。

章の basename を保持する設定やデータを足すときは ChapterRename::FOLLOWERS への登録が要ります。登録を忘れると vs rename が黙って参照を壊します。

♣ スキャフォールドを同期する

lib/project_scaffold/ は vs new が展開する雛形です。**その正本はプロジェクトルートにあります。** contents/ ・ stylesheets/ ・ images/ ・ config/ ・ templates/ などはルート側で開発・検証し、ruby scripts/copy_to_scaffold.rb で雛形へ複製します。

```
# ルートの stylesheets/ を編集してから
ruby scripts/copy_to_scaffold.rb
```

lib/project_scaffold/ を直接編集すると、次の同期で黙って消えます。逆に、ルートを直して同期を忘れると、著者の手元にだけ古いものが配られます。CSS を触ったら同期する、と覚えてください。

♣ 知見メモを引く

「CSS は正しいはずなのに効かない」「Kindle だけ崩れる」「並列ビルドで再現しない」といった、原因がエンジン側や基盤側にある問題は、たいてい一度は誰かが踏んでいます。docs/specs/NOTES.md がその索引です。

索引の区分	内容
恒常ガイドライン	記法を足す・設定キーを足す/やめる・章名を保持するデータを足すときの規範
知見メモ	CSS の落とし穴、Kindle の制約、ビルドの枝をまたぐ処理、Type 3 フォント、端末出力、組版の改行
検討メモ	まだ決めていないことの記録。着手するときの出発点

索引の各項目は「いつ引くか」で書かれています。探しものの名前ではなく、症状から引いてください。新しく -notes.md や -guidelines.md を書いたら、ここへ 1 行足すのが運用です。

実装済みの仕様書は `docs/archives/` へ移ります。「実装されているか」の答えは `docs/specs/STATUS.md` が持っており、置き場所から判断するものではありません。

6 テスト

Minitest を使っています。実ビルドを伴う重いスイートは専用タスクに分かれています。

```
bundle exec rake test
```

いずれも `rake` に続けて実行します。

タスク	内容
`test`	通常のユニットテスト（実ビルドなし）
`test:standard`	MIT 経路（`StandardProvider`）を強制実行
`test:versions`	対応する複数の Ruby で実行
`test:layout`	判型・ページレイアウトの統合テスト
`test:targets`	出力ターゲットの整合性（単体・複合を突き合わせ）
`test:type3`	Type 3 フォントの混入検証
`test:kindle`	Kindle 変換の検証
`test:notation`	記法網羅（CommonMark / GFM の記法が実ビルドで失われないか）
`test:manual`	マニュアル実ビルドの回帰検査。本書自身の原稿も検査します
`test:package`	パッケージング E2E（gem build から隔離インストールまで）
`test:canary`	依存カナリア（Vivliostyle CLI 最新版での破壊検知）
`test:sweep`	テストの作業ディレクトリ（`$TMPDIR/vs-*`）の残骸を掃除
`test:release`	リリース前の総点検

`test:release` が回すのは `sweep` → `test` → `standard` → `layout` → `targets` → `notation` → `manual` → `package` の順で、**全部入りではありません**。`canary` は上流の更新で落ちるものなので、リリースの可否とは切り離してあります。`type3`・`kindle`・`versions` は成果物を実機や外部ツールで見る性格のもので、必要なときに個別に走らせます。

`test:manual` はマニュアル（本書）を実ビルドするので、**原稿の文章品質もここで見ます**。`vs lint` の指摘が 1 件でも残っていれば、リリース前に赤くなります。

外部ツールに依存するテストはモックしており、ツール未導入の環境でも実行できます。

> `vivlio-starter-pdf` を入れた開発機では `rake test` は Enhanced 経路しか通りません。MIT 経路の回帰を捉えるには `rake test:standard` を併せて実行してください。本体のテストが AGPL の HexaPDF に依存しないよう、PDF は Prawn で作り pdf-reader で検査します。

♣ 堅牢性テスト

「想定外の入力・環境・操作」に対する振る舞いは `test/vivlio_starter/robustness/` に集約しています。カタログに登録済みのファイルがない場合の継続、パストラバーサルを含む画像パス、`vs new` 中断時の後始末、`vs lint --fix` 中断時のファイル保全、YAML の危険なタグの拒否、SIGINT/SIGTERM の終了コード、並列ビルドの排他などが対象です。

環境依存のテスト（書き込み権限・外部コマンド不在・シグナル送信）は、条件を満たさないときは理由を示して `skip` します。

7 堅牢性・セキュリティ設計

執筆プラットフォームとして、想定外の入力に対して「静かに壊れる」のではなく**明示的に通知して安全に停止する**ことを原則としています。

♣ 防衛線の階層

層	担当範囲	代表的な機構
入力検証	`book.yml` / `catalog.yml` / `data/*.yml`	`Common.ensure_configured!`（必須ファイルの検査・廃止キーの案内）、各ローダの `safe_load`
静的解析	原稿 Markdown	`LinkImageValidator.scan_dangerous_schemes`（`file://` / `javascript:`）
実行前提	コマンド実行の前提条件	`cli/guards/` の Guard と Check
外部コマンド	`vivliostyle` / `rsvg-convert` / ImageMagick ほか	`Common.external_command_available?`、`Common.ensure_external_command!`、`Common.run_svg_converter!`
同時実行制御	同一プロジェクトの多重ビルド	`BuildLock`（`flock` ベース）
シグナル	Ctrl+C / SIGTERM	`CLI.start` の三段 rescue
副作用保護	`vs new` 展開中 / `vs lint --fix` 中の中断	展開のクリーンアップ、一時ファイル経由の非破壊書き換え

♣ YAML の扱い

すべての YAML 読み込みで `YAML.safe_load` 系のみを使い、`permitted_classes` を明示します。`catalog.yml` は空（最小）、`data/*.yml` は実用上必要な範囲に限ります。`aliases: true` はプリセットの継承に使うため許可しています。`Psych::DisallowedClass` は素通しせず、ファイルパスとタグ名を含む日本語のメッセージへ変換して通知します。

♣ 常時有効とオプトアウトの区別

危険なスキームの検出は、どの設定でも無効化されません。外部 URL の確認のように切り替えられる検証もありますが、**セキュリティ保護は選択肢から外す**ことで、うっかり無効化による事故を防いでいます。

♣ 堅牢化を足すときの指針

1. **上流で例外を変換する。生のスタックトレースではなく、原因と次の一手を含む日本語のメッセージにする**
2. **原因特定に必要な情報を含める。ファイルパス・行番号・該当値・推奨アクション**
3. **回帰テストを `robustness/` に置く**
4. **常時有効か、オプトアウト可能かを区別する。セキュリティは常時有効、品質チェックは選択可能**
5. **環境依存のテストは理由付きで `skip` する**

8 検証済みの動作環境

項目	対応範囲
Ruby	3.4 以上（3.4.10 / 4.0.7 で検証）
Node.js	26 系
OS	macOS（`--fix` による自動導入は Homebrew 前提）

Vivliostyle 関連のパッケージは `package.json` で管理しています。

パッケージ	バージョン	役割
`@vivliostyle/cli`	11.3.3	CSS 組版エンジン CLI。`vs build` から呼び出す
`@vivliostyle/vfm`	2.7.2	Markdown から HTML への変換

パッケージ	バージョン	役割
`@vivliostyle/core`	2.45.1	レンダリングエンジン（CLI に内包）

`vs build` はグローバルに導入された `vivliostyle` コマンドを呼びます（`vs doctor --fix` が導入します）。11 系では脚注まわりの組版が大きく改善されています。最新版は npmjs.com/@vivliostyle/cli [1] で確認できます。

外部ツール（Ghostscript・ImageMagick・Inkscape・qpdf・MeCab ほか）の要否と導入は `vs doctor` が診断します。入稿用 PDF の導出には qpdf 11 以上が必要です。

本章で触れた設計の意図や実装の経緯は `docs/specs/` の仕様書と知見メモに記録されています。公開リポジトリの docs/specs [2] から読めます。フォークして開発するときは、変更前に該当する文書を確認し、実装後は `CHANGELOG.md` を更新してください。

---

1. https://www.npmjs.com/package/@vivliostyle/cli
2. https://github.com/Atelier-Mirai/vivlio-starter/tree/master/docs/specs